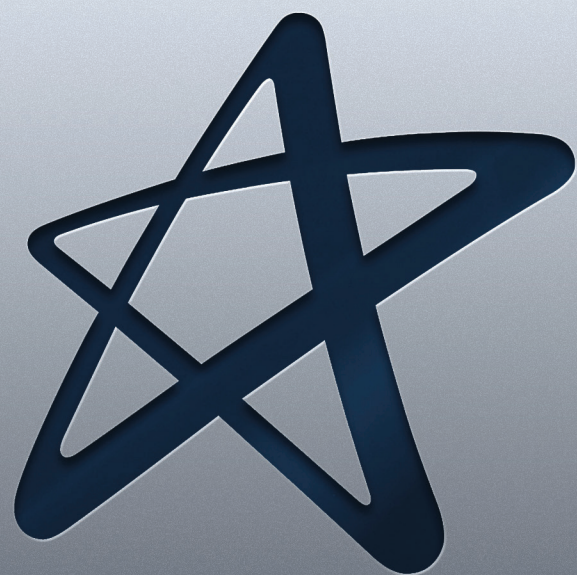


Compiladores e Interpretadores



Cruzeiro do Sul Virtual
Educação a distância

Material Teórico



Conceitos Básicos e Visão Geral dos Processos de Compilação e Interpretação

Responsável pelo Conteúdo:

Prof. Dr. Cleber Silva Ferreira da Luz

Revisão Textual:

Prof. Esp. Claudio Pereira do Nascimento

UNIDADE

Conceitos Básicos e Visão Geral dos Processos de Compilação e Interpretação



- Introdução;
- Processo de Tradução;
- Processo de Compilação de Uma Linguagem;
- Conclusão e Resumo da Unidade.



OBJETIVO DE APRENDIZADO

- Apresentar todos os conceitos básicos sobre o processo de compilação e interpretação.



Orientações de estudo

Para que o conteúdo desta Disciplina seja bem aproveitado e haja maior aplicabilidade na sua formação acadêmica e atuação profissional, siga algumas recomendações básicas:



Assim:

- ✓ Organize seus estudos de maneira que passem a fazer parte da sua rotina. Por exemplo, você poderá determinar um dia e horário fixos como seu “momento do estudo”;
- ✓ Procure se alimentar e se hidratar quando for estudar; lembre-se de que uma alimentação saudável pode proporcionar melhor aproveitamento do estudo;
- ✓ No material de cada Unidade, há leituras indicadas e, entre elas, artigos científicos, livros, vídeos e *sites* para aprofundar os conhecimentos adquiridos ao longo da Unidade. Além disso, você também encontrará sugestões de conteúdo extra no item **Material Complementar**, que ampliarão sua interpretação e auxiliarão no pleno entendimento dos temas abordados;
- ✓ Após o contato com o conteúdo proposto, participe dos debates mediados em fóruns de discussão, pois irão auxiliar a verificar o quanto você absorveu de conhecimento, além de propiciar o contato com seus colegas e tutores, o que se apresenta como rico espaço de troca de ideias e de aprendizagem.

Introdução

Nesta disciplina, estudaremos o processo de compilação e de interpretação das linguagens de programação. Atualmente, boa parte das linguagens de programação são compiladas ou interpretadas. Estudaremos também todos os conceitos envolvidos no processo de compilação e interpretação de uma linguagem.

Segundo Louden (2004), compiladores e interpretadores são programas de computadores que traduzem uma linguagem para outra, isto é, traduzem uma linguagem de alto nível, onde um ser humano, com o mínimo de conhecimento em programação, conseguirá compreender uma linguagem que só computadores entendem.

Processo de Tradução

Computadores trabalham (entendem) com zeros e uns. Todo trabalho que o computador realiza é através do famoso **código binário**. Um código binário é um código formado somente por zero e um. Ao se criar um *software*, isto é, criar um programa, utilizamos uma linguagem de programação e criamos um código fonte que contém todas as instruções que o programa deveter. Assim, o código fonte é composto por instruções que o *software* (programa) deve realizar, todavia, essas instruções são descritas em alto nível, onde qualquer ser humano consegue entender, só que computadores entendem somente zeros e uns. E agora? Para resolver essa situação, é necessário realizar uma **tradução** do que foi escrito em alto nível para o baixo nível (zeros e uns). A Figura 1 ilustra o processo de Tradução.



Figura 1 – Tradução

Há duas formas de realizar essa tradução: através do compilador e através do interpretador e o compilado. Compilador e tradutor possuem o mesmo objetivo, o de realizar a tradução de alto nível para o baixo nível. O objetivo é o mesmo, porém, a forma como a tradução se realizada é diferente. Um compilador realiza a tradução e no final gera um executável enquanto que o interpretador realiza a tradução em tempo de processamento, ou seja, as instruções são traduzidas e executadas logo em seguida. Já no compilador, as instruções são traduzidas e executadas posteriormente.

Atualmente, as linguagens de programação são compiladas ou interpretadas, como exemplo de linguagens interpretadas, é possível citar Ruby e R. linguagens interpretadas. A Figura 2 ilustra o processo de tradução do interpretador.

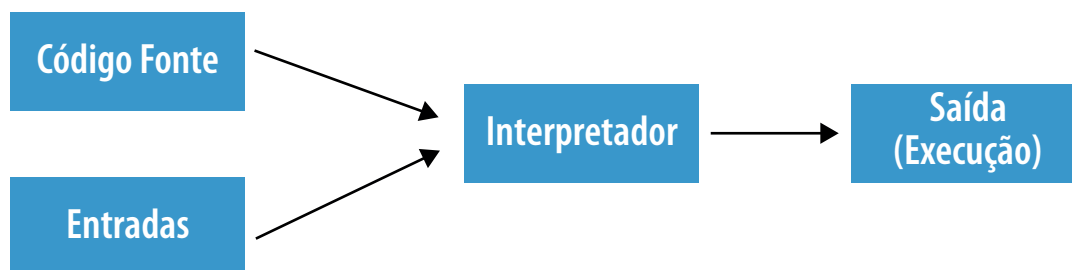


Figura 2 – Interpretação de uma linguagem

A seguir, são apresentadas vantagens e desvantagens de um interpretador.

Vantagens do interpretador:

- Na interpretação, as correções e alterações são realizadas de forma mais rápida;
- Não há um executável, o interpretador traduz e executa em tempo de processamento.

Desvantagens do interpretador:

- Pelo fato da execução ser realizada junto com a execução, a execução do programa é mais lenta;
- O interpretador precisa ler o código novamente para executar o programa.

Já um compilador recebe como entrada um código fonte, realiza a tradução e gera um executável. Como exemplo de linguagens compiladas, podemos citar Java e C. O processo de compilação é ilustrado na Figura 3.



Figura 3 – Compilação de uma linguagem

Vantagens do compilador:

- O acesso ao código compilado é mais rápido, pois executa as instruções após todo o processo de compilação;
- Permite otimização do código;
- Executa o programa somente se não existir erros.

Desvantagens do compilador:

- O processo de compilação é mais complexo e composto por várias sub etapas;
- Se houver erros, o código é novamente compilado.

Antes de entrarmos a fundo no processo de compilação, é preciso ter bem claro em nossas mentes os seguintes conceitos:

- **Símbolos:** são elementos mínimos que compõem uma linguagem. São exemplos de símbolos as letras, os dígitos, entre outros. Símbolos podem ser **ordenados**. Por exemplo, tomando as letras do alfabeto, tem-se uma ordenação

$A < B < C < D < E \dots Z$. A principal utilidade dos símbolos está na possibilidade de **usá-los como elementos em definições de linguagens**.

- **Sentença (ou palavra):** sequência finita de símbolos. Exemplo, sejam C, A, R, R e O **símbolos**, então CARRO é uma sentença. Uma sentença pode ser vazia, constituída por nenhum símbolo. Uma sentença vazia é representada por ϵ .
- **Alfabeto:** um conjunto de símbolos. Um alfabeto, denotado por V , é um **conjunto finito de símbolos**, assim, considerando os símbolos dígitos, letras e etc. **Exemplo:** alfabetos $V_{\text{binários}} = \{0,1\}$; $V_{\text{vogais}} = \{a, e, i, o, u\}$.
- **Linguagem:** forma de comunicação usada por sujeitos de uma determinada comunidade. Uma linguagem é um conjunto de símbolos e regras que, combinados, geram sentenças sintaticamente corretas.
- **Gramática:** pode ser considerada como a forma de representar as regras para formação de uma linguagem.

Esses conceitos básicos são de suma importância para o entendimento de compiladores. Sabe por quê? Por exemplo, aplicando esses conceitos para uma linguagem de programação (pode-se considerar JAVA, por exemplo) temos, portanto:

- **Alfabeto:** $\{i, f, w, h, i, l, e, -, *, 1, 2, 5, 9\}$;
- **Símbolos:** $\{2, 5, w, f\}$;
- **Sentença ou palavra:** $\{for, else, 853, -1\}$;
- **Linguagem:** $\{if, for, while, -1, 63\}$.

Entendendo esses conceitos básicos de forma bem clara, podemos entrar no mundo dos compiladores. A próxima seção explica com detalhes o processo de compilação de uma linguagem.

Processo de Compilação de Uma Linguagem

O processo de interpretação que uma linguagem realiza é muito simples. Já o processo de compilação é complexo e realizado por etapas. A compilação de uma linguagem é composta por 2 etapas:

- Análise;
- Síntese.

A primeira etapa a ser realizada é a de análise. Nessa etapa, o código fonte é analisado por 3 subetapas. Cada subetapa realiza uma análise no código para verificar se o conteúdo do código fonte condiz com a linguagem. A etapa de síntese consiste na geração do executável. Uma vez que a tradução esteja correta, é gerado

o código executável. A etapa de síntese também é conhecida simplesmente como **geração de código**.

A etapa de análise é formada pelas subetapas:

- Análise léxica;
- Sintática;
- Semântica;

A etapa de síntese é formada pelas subetapas:

- Geração de código intermediário;
- Otimização;
- Geração de código.

A Figura 4 ilustra o processo de compilação.

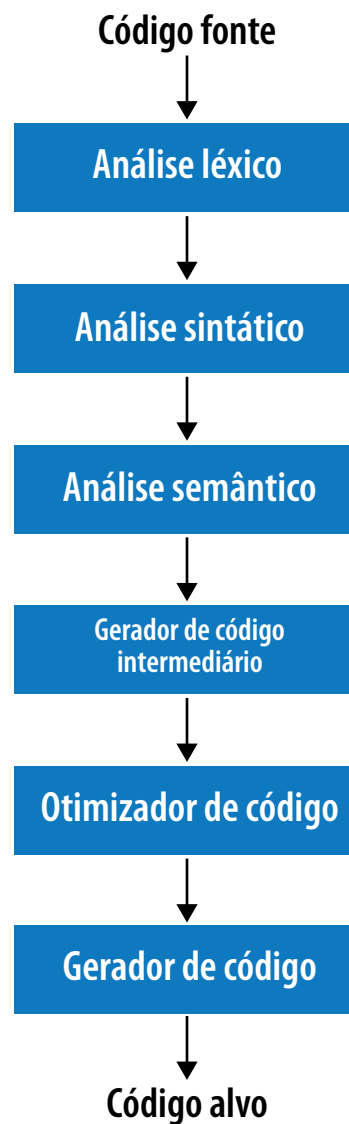


Figura 4 – Processo de compilação

Análise Léxica

O analisador léxico lê o código fonte como um arquivo de caracteres e o separa em **elementos**, tais elementos são como palavras de uma linguagem natural. Cada **elemento** é formado por uma sequência finita de caracteres que representa uma **unidade de informação** do **programa-fonte**. Por exemplo, **palavras reservas**, como **for** e **else** que são formadas por um número fixo de caracteres; os **identificadores**, que são formados por um número dinâmico de caracteres, a quantidade de caracteres de um identificador é definida pelo usuário e usualmente composta por letras e números começando por letras; **símbolos especiais**, tais como “<”, “>”, “>=”. Em cada caso, um elemento representa um determinado padrão de caracteres reconhecidos por um padrão de caracteres reconhecidos pelo analisador léxico à medida que os caracteres são fornecidos.

Em suma, o analisador léxico encara o código fonte como um arquivo de caracteres e identifica cada elemento significativo aceito na linguagem de programação em que o código fonte foi escrito, por exemplo, em um código fonte “JAVA”. O analisador léxico então identifica todos os elementos significativos aceitos em JAVA. Essa análise léxica também é chamada de varredura.

O analisador léxico, também chamado de analisador de varredura, encara o código fonte como um arquivo de texto e o percorre a fim de encontrar **padrões**. Para exemplificar, considere a linguagem de programação C, que realiza uma declaração de variável de seguinte maneira:

```
int x = 12;
```

A análise léxica percorre o texto para encontrar padrões. Assim, por exemplo, quando o analisador léxico encontrar os caracteres **i**, **n**, **t**, nesta ordem, será encontrado um padrão, portanto, os caracteres i, n, t casam com padrão da palavra int. Todavia, vamos supor que o programador escreveu a seguinte instrução no código fonte:

```
ˆint y = 8;
```

O analisador léxico encontrou o caractere **ˆ** (**agudo**), seguido dos caracteres i,n,t, mas tais caracteres não casam com nenhum padrão. Portanto, é gerado um erro léxico.

O analisador léxico realiza um processo de varredura no código fonte. Segundo Louden (2004), é de responsabilidade do sistema de varredura **ler caracteres** do código fonte e **organizá-los em unidade lógicas** que serão analisados por outros componentes do compilador.

As unidades lógicas geradas pela varredura são denominadas **tokens**. Organizar caracteres em *token* é semelhante a organizar caracteres em palavras para uma sentença em português e decidir quais palavras usar. Podemos dizer que a varredura se assemelha à atividade de soletrar.

Em uma maneira simples, podemos dizer que o processo de varredura lê os caracteres do código texto fonte, verificando se eles pertencem ao alfabeto da

linguagem, identificando os *tokens*. Nessa verificação, comentários e espaços em brancos são ignorados.

Os *tokens* constituem classes de símbolos. Há diversas classes de *Tokens*, como por exemplo, a classe das **palavras reservadas**, como **FOR** e **IF** que representam as cadeias de caracteres “**for**” e “**if**”. Como um segundo exemplo, podemos citar a classe de **símbolos especiais**, como os símbolos aritméticos “**mais**” e “**menos**”, que representam os caracteres “**+**” e “**-**”. Existe, ainda, uma classe para representar cadeia de múltiplos caracteres. Alguns, por exemplo, são **NUM** e **ID**, que representam **números** e **identificadores**.

Quando se projeta um compilador, é necessário definir a **Tabela de Símbolos** que serão aceitos pela linguagem. Segundo Louden (2004), a análise léxica é responsável por varrer o código fonte e identificar os *tokens*. Também é de responsabilidade do analisador léxico iniciar a tabela de símbolos que pode ser considerada como uma estrutura de dados gerada pelo compilador, visando manter informações que são utilizadas por ele. A tabela de símbolo faz a associação dos atributos (tipo, limites no caso de vetores e número de parâmetros no caso de funções) aos nomes que foram definidos pelo programador.

A construção da tabela de símbolos começa durante a execução da análise léxica que, por sua vez, é responsável por inicializar a tabela de símbolos. Assim, quando o analisador léxico encontrar um identificador pela primeira vez, ele o armazena na tabela de símbolos. O analisador léxico obtém os *tokens* do código fonte e passa para o analisador sintático (LOUDEN, 2004).

Na análise léxica, há três termos muito importantes, **Padrão**, **Lexema** e **Token**. Esses três termos são apresentados com detalhes na Tabela 1.

Tabela 1

Padrão	Podemos dizer que Padrão é a maneira (forma) como os símbolos de uma cadeia de caracteres podem estar, isto é, podem assumir. Assim, considerando uma palavra reservada, é a sequência de caracteres que formam a palavra reservada.
Lexema	Já um lexema pode ser considerado como uma sequência de caracteres no código fonte que casa com um padrão para um <i>Token</i> . Um lexema também pode ser considerado como uma instância de <i>Token</i> .
Token	<i>Token</i> são símbolos léxicos reconhecidos através de um padrão. Esse nome é um símbolo abstrato, representando um tipo de unidade léxica. Por exemplo, palavra-chave, identificador, número e entre outros.

A tabela abaixo apresenta exemplos de **Padrão**, **Lexema** e **Token**.

Tabela 2

<i>Token</i>	Padrão	Lexema	Descrição
<while>	Sequência dos caracteres w, h, i, l, e	while, While, WHILE	Palavra reservada laço
<if>	Sequência dos caracteres i,f	if, IF, iF, If	Palavra reservada – tomada de decisão
<Soma>	Sequência dos caracteres +	+	Símbolo aritmético soma
<id>	Sequência dos caracteres s, o, m, a	soma	Identificador de variável
<Number>	Sequência dos 2585	2585	Número

Para melhor entendimento, considere a seguinte instrução:

```
printf ("Total é" + soma)
```

Nessa instrução, `printf` e `soma` são lexemas associando o padrão para o *token id* (identificadores, pois identificam a variável `soma` e a função `printf`) e `"Total"` é um lexema que associa o padrão para o *token literal*.

Como já mencionado anteriormente, os tokens são símbolos léxicos reconhecidos através de um padrão. *Tokens* são classificados em dois grupos:

- **Tokens simples:** não têm valor associado, pois a classe do *token* já a descreve.
Exemplo: palavras reservadas, operadores, delimitadores: `<if,>`, `<else,>`, `<+,>`.
- **Tokens com argumento:** têm valor associado e correspondem a elementos da linguagem definidos pelo programador. **Exemplo:** identificadores, constantes numéricas - `<id, 3>`, `<número, 10>`, `<literal, Olá Mundo>`.

Implementação de analisadores léxicos

Existem várias ferramentas e métodos para se implementar um analisador léxico, entre eles, encontra-se o Lex, desenvolvido por Eric Schmidt e Mike Lesk. O Lex é um programa que permite gerar analisadores léxicos, habitualmente é usado em conjunto com o YACC, um programa que permite gerar analisadores sintáticos.

Apesar do Lex ser um *software* de propriedade particular, há várias versões do Lex de código aberto, uma delas é JFLEX que é baseada em JAVA.

Um arquivo Lex é dividido em 3 seções, separadas por linhas que contêm somente dois símbolos de porcentagem, como mostrado no exemplo a seguir:

definições

%%

regras

%%

sub-rotinas

Vamos considerar o JFLEX para exemplificar o Lex. Na seção de **definições** são definidos os macros e são importadas as bibliotecas JAVA. A seção de **regras** associa padrões com instruções JAVA, padrões escritos na forma de expressões regulares.

No processo realizado pelo analisador léxico, quando é encontrado um texto que casa com um padrão, as instruções em JAVA associadas são executadas. Podemos dizer que a tentativa do casamento é gulosa, isto é, no caso de dois padrões diferentes casando a mesma entrada, o maior deles será usado. Já a seção de **sub-rotinas** contém blocos de código JAVA que serão apenas copiados ao arquivo final. Nas unidades futuras, o JFLEX será mais detalhado.

Em suma, o JFLEX lê um arquivo contendo expressões regulares para encontrar padrões e gerar instruções em JAVA que realizam a análise léxica. O JFLEX lê o código fonte e identifica os *tokens*.

Análise sintática

Nessa análise é determinada a sintaxe ou estrutura de um programa (LOUDEN, 2004). Na análise sintática é verificado a estrutura das instruções, ou seja, é verificado se os elementos da instrução estão na ordem correta. Podemos falar ainda que o analisador sintático visa verificar se uma determinada gramática, com uma sequência de símbolos terminais (Frase), é uma frase válida. Para exemplificar esta análise, considere a instrução a seguir:

```
int total = 12;
```

na análise sintática é verificado a ordem dos elementos (int, total, =, 12, ;). Vamos supor que o programador digitou a seguinte instrução no código fonte;

```
total int 12 =;
```

os elementos dessa instrução estão em ordem incorretas. O analisador sintático gera um erro sintático para esta instrução. É importante observar que os elementos existem na linguagem e foram casados com um padrão, todavia, eles estão em ordem incorreta. Podemos dizer, ainda, que o analisador sintático representa uma instrução através de uma estrutura de dados. Habitualmente, esta estrutura é uma **árvore** que ajuda a capturar a hierarquia da instrução. Por exemplo, $8 - 4 / 2$ poderia ser expressado pela árvore apresentada na Figura 5.

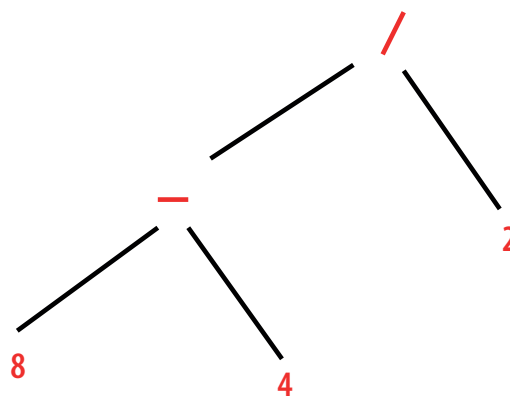


Figura 5 – Árvore para a expressão $8 - 4 / 2$

O analisador sintático será estudado com mais detalhes na unidade III.

Implementação de um analisador sintático

Assim como o analisador léxico, há vários métodos e mecanismos para implementar um analisador sintático. Entre eles encontra-se o YACC, desenvolvido por Stephen C. Johnson. O YACC é um programa que permite criar analisadores

sintáticos que fornece sentido sintático a um determinado código fonte, baseado numa gramática formal escrita em uma forma similar ao formalismo de *Backus-Naur*. Habitualmente, o YACC é utilizado em conjunto com o LEX.

Análise semântica

A análise semântica é realizada por um analisador semântico. Nesse processo, a análise realizada verifica se as instruções realmente fazem sentido. No processo de compilação, a última análise realizada é a análise semântica, na qual é verificada se as sentenças que o usuário escreveu no código fonte realmente fazem sentido. Considere a instrução a seguir como exemplo:

```
int y = "nome";
```

esta instrução iria passar pelo analisador léxico, pelo analisador sintático, mas não pelo analisador semântico. Isso porque não faz sentido colocar um *String* dentro de uma variável do tipo inteiro. O compilador iria gerar um erro semântico.

Podemos dizer que na análise semântica não existe uma maneira formal de realizá-la porque depende muito da linguagem, por exemplo: se essa instrução fosse em R, não teria problema, pois, quando se declara variável em R, não se define o tipo de dado que aquela variável poderá aceitar. Assim, instruções como $X = 12$; ou $X = \text{"Nome"}$ estão corretas em R. A análise semântica é estudada com detalhes na unidade IV.

Geração de Código

Após realizar a etapa de análise, a próxima etapa será gerar o executável. Segundo Loudon (2004), a fase de geração de código é a fase mais complexa de um processo de compilação, pois depende não apenas das características da linguagem fonte, mas também de informações adicionais, como informações sobre a arquitetura do computador e também do sistema operacional.

A fase de geração de código é dividida em três etapas;

- Geração de código intermediário;
- Otimização;
- Geração de código.

A geração de código é uma fase complexa e, por esse motivo, habitualmente, a geração de código é realizada em vários passos que usam diversas estruturas de dados e frequentemente requerem alguma forma de código abstrato, denominado **código intermediário**.

Você deve estar se perguntando, por que gerar o código intermediário? Não poderia gerar direto o código executável? Qual a vantagem do código intermediário? Vamos responder essas perguntas com respostas extremamente fáceis. Primeiro,

com a geração de código intermediário é possível otimizar o código, deixando-o mais rápido e menor. Segundo, com a geração de código intermediário é possível explorar mais a arquitetura do computador, isso fará com que a execução do código gerado no final aproveite todos os recursos possíveis da arquitetura do computador. Vale lembrar que cada computador possui uma arquitetura diferente.

A etapa de **otimização** é responsável por melhorar a velocidade e/ou tamanho do código gerado. Para tal, são coletadas informações adicionais que irão permitir o ajuste do código gerado para fazer um melhor uso das características específicas do computador. Tais informações adicionais são, por exemplo, quantidade de registradores, modos de endereçamento e memória de armazenamento intermediário. Essa etapa trabalha com o código intermediário a fim de melhorá-lo.

Para exemplificar tais otimizações, considere o código apresentado na Figura 6.

```
public class Teste {  
    public static void main(String[] args) {  
        int a = 0;  
        for(int i = 0; i < 100; i++){  
            a++;  
        }  
    }  
}
```

Figura 6 – Trecho de código para exemplificar a otimização

Fonte: Acervo do conteudista

Observe neste código a variável *a*. Ela é declarada antes do *for* e incrementada dentro do *for*, todavia, essa variável não é utilizada. É muito provável que, quando este código for compilado, o compilador não irá gerar o código binário para a instrução de incremento dentro do *for*, pois não faz sentido gerar o código binário para ela, sendo que essa instrução não é utilizada. Assim, o código executável gerado pelo compilador ficará mais rápido.

No processo de otimização, são realizadas otimizações semelhantes a essa.

Conclusão e Resumo da Unidade

Atualmente, as linguagens de programação são interpretadas ou compiladas. Compiladores e interpretadores são programas de computadores que traduzem uma linguagem para outra, isto é, traduzem uma linguagem de alto nível, onde um ser humano com o mínimo de conhecimento em programação consegue entender para uma linguagem que só computadores entendem.

O processo de compilação é composto por duas etapas; “análise” e “síntese”. A etapa de análise corresponde a análise do código fonte enquanto que a etapa

de síntese corresponde a etapa em que o executável é gerado. A etapa de análise é composta pelas análises Léxica, Sintática e Semântica. Já a etapa de síntese é composta pelas subetapas de geração de código intermediário, otimização e geração de código.

A compreensão do processo de compilação de uma linguagem é de suma importância para um profissional da área de computação. O entendimento do processo de compilação ajuda a entender o funcionamento de um compilador e como as instruções em um código fonte são processadas.

Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:

Livros

Construindo Compiladores

KEITH D. C.; TOREZON, L. **Construindo Compiladores**. 2ª Edição. Editora Campus, Elsevier.

Leitura

Basics of Compiler Design

MOGENSEN, T. A. *Basics of Compiler Design*, 2010.

<http://bit.ly/2LorUn2>

Lexical Analysis and Parsing using C

PREISS, B., *Lexical Analysis and Parsing using C*, 2004.

<http://bit.ly/2oN4MXI>

Compiler Construction

WAITE, W. M.; GOOS, G., *Compiler Construction*, 1996.

<http://bit.ly/2LoT2Cu>

Referências

AHO, A. V. **Compiladores: Princípios, Técnicas e Ferramentas**. 2. ed. São Paulo: Pearson. (e-book)

APPEL, A. W. **Modern Compiler Implementation. In Java**. 2. ed. New York: Cambridge University Press, 2002.

BERGMANN, S. D. **Compiler Design: Theory, Tools, and Examples**, 2016. Disponível em: <<http://www.cs.cmu.edu/~aplatzer/course/Compilers/waitegoos.pdf>>. Acesso em: 20 set. 2019. (e-book)

LOUDEN, K. C. **Compiladores: Princípios e Práticas**. São Paulo: Pioneira Thomson Learning, 2004.

MENEZES, P. B. **Linguagens formais e autômatos**. 6. ed. São Paulo: Bookman, 2011.

PRICE, A. M. A. **Implementação de Linguagens de Programação: Compiladores**. 2. ed. Porto Alegre: Sagra Luzzatto, 2001.

RAMOS, M. V. M.; VEGA, I. S.; JOSE NETO, J. **Linguagens formais: teoria, modelagem e implementação**. Porto Alegre: Bookman, 2009. 656 p.



Cruzeiro do Sul
Educatonal